

Section 11 - Developer Reference

Architecture Overview

SalesOps AI is a serverless REST API deployed with AWS SAM. All Lambda functions share a single Node.js 24 runtime under the LabRole execution role. API Gateway provides the HTTP edge. Cognito authenticates users. DynamoDB stores all application data. SQS manages delayed exam issue release. Secrets Manager stores the OpenAI API key.

Base URL pattern:

```
https://<api-id>.execute-api.us-east-1.amazonaws.com/dev
```

The stage name dev matches the StageName CloudFormation parameter default.

Auth Model

Every request flows through one of three auth layers:

Layer	Mechanism	Routes
Public	No auth	/health, /auth/signup, /auth/confirm, /auth/resend-confirmation, /auth/signin, /auth/refresh, /auth/forgot-password, /auth/forgot-password/confirm
Cognito authorizer	API Gateway validates Cognito ID token in Authorization: Bearer <idToken> header	All /auth/me, /users, /personas, /scenarios, /dashboard, /exam routes
Role guard	Lambda reads Users DynamoDB table after Cognito auth	Manager routes require role=manager, rep routes require role=rep, all require status=ACTIVE

Response Envelope

All handlers return JSON. Success and error bodies share the same Content-Type header.

```
HTTP 200 / 201 / 204
Content-Type: application/json
Access-Control-Allow-Origin: *
{ <route-specific fields> }
```

Error body:

```
HTTP 4xx / 5xx
Content-Type: application/json
{ "message": "<human-readable error string>" }
```

Data Types

The following types appear repeatedly in request and response bodies.

UserProfile

Returned by auth and user management routes.

	Type	Description		
	string	Cognito sub. Primary key in the Users DynamoDB table.		
	string	User email address as entered.		
	string	Normalized lowercase email. Used for lookup via EmailIndex GSI.		
	string	Display name.		
	"rep" \	"manager"	Application role. Controls which routes are accessible.	
	"PENDING_CONFIRMATION" \	"ACTIVE" \	"SUSPENDED"	Account lifecycle status
	ISO 8601 string	Account creation timestamp.		
	ISO 8601 string	Last modification timestamp.		

Persona

Field	Type	Description
personaId	string	Prefixed UUID (persona_<uuid>).
name	string	Display name for the persona.
description	string	Purpose or background description.
behaviorNotes	string	Behavioral traits used during issue generation.
status	"ACTIVE"	Always ACTIVE for all current personas.
createdAt	ISO 8601 string	Creation timestamp.
updatedAt	ISO 8601 string	Last update timestamp.

ScenarioIssue

Issues embedded inside a Scenario object (in the Scenarios table) and copied into ExamSessions when a session starts.

	Type	Description		
	string	Prefixed UUID (issue_<uuid>).		
	string	Reference to the source persona.		

	Type	Description		
	string	Fictional customer name.		
	string	Email subject line.		
	string	Full customer message body.		
	"EASY" \	"MEDIUM" \	"HARD"	Difficulty rating.
	"DRAFT"	All scenario issues are DRAFT.		
	ISO 8601 string			
	ISO 8601 string			

Scenario

	Type	Description		
	string	Prefixed UUID (scenario_<uuid>).		
	string	Short name for the scenario.		
	string	Narrative description shown to reps before an exam.		
	string[]	Ordered list of persona IDs linked to this scenario.		
	integer	Target number of inbox issues. Range 1-20.		
	ScenarioIssue[]	Generated issues stored on the scenario item. Empty before generation.		
data	ISO 8601 string	Timestamp of last generation. Empty string if not generated.		
source	"OPENAI" \	"DEMO" \	""	Source of the last generation.
using	string	Non-empty when demo issues were used because OpenAI failed.		
	"DRAFT" \	"PUBLISHED" \	"ARCHIVED"	Lifecycle state.
	ISO 8601 string			
	ISO 8601 string			

ExamSession

Returned by the exam session routes.

Field	Type	Description	
sessionId	string	Prefixed UUID (session_<uuid>). Partition key of the ExamSessions table.	
scenarioId	string	Reference to the source scenario.	

Field	Type	Description	
title	string	Snapshot of the scenario title at session creation.	
description	string	Snapshot of the scenario description.	
durationSeconds	integer	Exam duration. Always 180.	
totalIssues	integer	Number of issues in the session.	
startedAt	ISO 8601 string	Session creation time.	
endsAt	ISO 8601 string	Session expiry time (startedAt + 180s).	
status	"ACTIVE" \	"ENDED"	Session lifecycle state.

ExamIssue

Returned inside pulse responses as visible issues.

	Type	Description		
	string	Issue ID copied from the scenario.		
	string	Fictional customer name.		
	string	Email subject.		
	string	Customer message body.		
	"EASY" \	"MEDIUM" \	"HARD"	Difficulty rating.
	"PENDING" \	"VISIBLE" \	"DONE"	Visibility and completion.
	integer	0-based release order.		
	ISO 8601 string	Scheduled release time.		
	ISO 8601 string	Actual time the issue became visible. Empty if not yet visible.		
	ISO 8601 string	Time the rep marked the issue done. Empty if not done.		
	ExamResponse[]	All responses the rep submitted for this issue.		

ExamResponse

Field	Type	Description
responseId	string	Prefixed UUID (response_<uuid>).
message	string	Rep response text. Maximum 4000 characters.
createdAt	ISO 8601 string	Response submission time.

Rubric

Weighted rubric scores returned inside an evaluation. All scores are integers 0-100.

Field	Type	Weight	Description
kindness	integer	25%	Empathy, warmth, patience, human tone.
professionalism	integer	25%	Business-safe wording, no blame, respectful confidence.
resolution	integer	25%	Directly solves or advances the customer issue.
clarity	integer	15%	Concise, structured, easy to understand.
helpfulIdeas	integer	10%	Proactive options or prevention ideas.

The weighted score formula:

```
score = round(kindness*0.25 + professionalism*0.25 + resolution*0.25 + clarity*0.15 + helpfulIdeas*0.10)
```

EvaluationIssue

Per-issue coaching data inside an evaluation.

Field	Type	Description
issueId	string	Issue ID matching the exam session issue.
subject	string	Issue subject copied from the exam.
score	integer	Per-issue score 0-100 from OpenAI.
notes	string[]	Specific coaching notes for this issue (max 8).
suggestedAnswerIdeas	string[]	Alternative answer ideas from OpenAI (max 8).

ExamEvaluation

Field	Type	Description
sessionId	string	Parent session ID.
status	"COMPLETED"	Always COMPLETED when returned.
score	integer	Final weighted score 0-100.
evaluatedAt	ISO 8601 string	Evaluation creation time.
rubric	Rubric	Five-dimension rubric scores.
aiNotes	string[]	Overall session coaching notes (max 8).
strengths	string[]	Observed positive behaviors (max 8).

Field	Type	Description
growthAreas	string[]	Priority improvement areas (max 8).
practiceIdeas	string[]	Actionable practice suggestions (max 8).
issues	EvaluationIssue[]	Per-issue coaching results.

Health

GET /health

Returns service liveness. No authentication required. Called by the smoke test script and by any monitoring tool.

Handler: src/handlers/health.js → exports.handler

Auth: None

Request parameters: None

Response: 200 OK

```
{
  "status": "ok",
  "service": "salesops-ai",
  "stage": "dev",
  "timestamp": "2026-05-27T10:00:00.000Z"
}
```

Field	Type	Description
status	"ok"	Always "ok" when the Lambda executes.
service	string	Value of the SERVICE_NAME environment variable.
stage	string	Value of the STAGE_NAME environment variable.
timestamp	ISO 8601 string	Current Lambda execution time.

Errors: None. Any non-200 response means API Gateway or Lambda execution failed at the infrastructure level.

Authentication

All auth handlers read USER_POOL_CLIENT_ID and USERS_TABLE_NAME from Lambda environment variables. If either variable is missing, the handler returns 500.

POST /auth/signup

Creates a Cognito user account and a DynamoDB user profile. The new profile is always role rep and status PENDING_CONFIRMATION.

Handler: src/handlers/auth.js → exports.signup

Auth: None

Request body (JSON):

Field	Type	Required	Constraints
email	string	yes	Normalized to lowercase and trimmed. Must be unique in Cognito.
password	string	yes	Minimum 8 characters, one uppercase, one lowercase, one digit.
fullName	string	yes	Stored as-is after trimming whitespace.

Example request:

```
{
  "email": "riley@salesops.ai",
  "password": "Secure123",
  "fullName": "Riley Rep"
}
```

Response: 200 OK

```
{
  "userId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "email": "riley@salesops.ai",
  "nextStep": "CONFIRM_EMAIL"
}
```

Field	Type	Description
userId	string	Cognito sub. Use this as the partition key in User's DynamoDB operations.
email	string	Normalized email stored in DynamoDB.
nextStep	"CONFIRM_EMAIL"	Always this value. Instructs the frontend to prompt for confirmation.

Errors:

Status	Condition
400	Missing email, password, or fullName. Invalid password format.
409	Cognito UsernameExistsException — account already exists.
429	Too many requests to Cognito.

Status	Condition
500	Cognito did not return a UserSub. Service misconfiguration.

POST /auth/confirm

Confirms a Cognito signup code and sets the DynamoDB profile status to ACTIVE. Safe to call again after the account is already confirmed.

Handler: src/handlers/auth.js → exports.confirm

Auth: None

Request body (JSON):

Field	Type	Required	Constraints
email	string	yes	Must match the email used during signup. Normalized to lowercase.
code	string	yes	6-digit confirmation code from Cognito confirmation email.

Example request:

```
{
  "email": "riley@salesops.ai",
  "code": "123456"
}
```

Response: 200 OK

```
{
  "status": "confirmed"
}
```

Errors:

Status	Condition
400	Missing email or code. Code mismatch or expired code.
429	Too many confirmation attempts.

POST /auth/resend-confirmation

Resends the Cognito confirmation code email. Call when the original code is lost or expired.

Handler: src/handlers/auth.js → exports.resendConfirmation

Auth: None

Request body (JSON):

Field	Type	Required
email	string	yes

Response: 200 OK

```
{
  "status": "sent"
}
```

Errors:

Status	Condition
400	Missing email.
429	Rate limit exceeded.

POST /auth/signin

Authenticates the user with Cognito using email and password. Returns Cognito tokens and the DynamoDB profile.

Handler: src/handlers/auth.js → exports.signin

Auth: None

Request body (JSON):

Field	Type	Required	Constraints
email	string	yes	Normalized to lowercase.
password	string	yes	

Example request:

```
{
  "email": "dana@salesops.ai",
  "password": "Secure123"
}
```

Response: 200 OK

```
{
  "idToken": "<cognito-id-token>",
  "accessToken": "<cognito-access-token>",
  "refreshToken": "<cognito-refresh-token>",
  "expiresIn": 3600,
  "user": { "<UserProfile>" }
}
```

Field	Type	Description
idToken	string	JWT. Valid for 1 hour. Send in Authorization: Bearer <idToken> on protected routes.
accessToken	string	Cognito access token. Not used by API routes.
refreshToken	string	Valid for 30 days. Use in POST /auth/refresh.
expiresIn	integer	Seconds until idToken and accessToken expire. Always 3600.
user	UserProfile	Full DynamoDB profile.

Errors:

Status	Condition
400	Missing email or password.
401	Email or password is incorrect.
403	Email confirmed but account status is not ACTIVE.
403	Email not confirmed (UserNotConfirmedException).

POST /auth/refresh

Exchanges a refresh token for a new ID token and access token. Does not return a new refresh token.

Handler: src/handlers/auth.js → exports.refresh

Auth: None

Request body (JSON):

Field	Type	Required
refreshToken	string	yes

Response: 200 OK

```
{
  "idToken": "<new-cognito-id-token>",
  "accessToken": "<new-cognito-access-token>",
  "expiresIn": 3600
}
```

Errors:

Status	Condition
400	Missing refreshToken.
401	Refresh token expired or revoked.

POST /auth/forgot-password

Starts the Cognito forgot-password flow. Sends a reset code to the account email.

Handler: src/handlers/auth.js → exports.forgotPassword

Auth: None

Request body (JSON):

Field	Type	Required
email	string	yes

Response: 200 OK

```
{
  "status": "sent"
}
```

Note: Returns 200 even if the email does not exist in Cognito, to prevent account enumeration.

Errors:

Status	Condition
400	Missing email.
429	Rate limit exceeded.

POST /auth/forgot-password/confirm

Completes the forgot-password flow by submitting the reset code and setting a new password.

Handler: src/handlers/auth.js → exports.confirmForgotPassword

Auth: None

Request body (JSON):

Field	Type	Required	Constraints
email	string	yes	Normalized to lowercase.
code	string	yes	Reset code from the Cognito email.
password	string	yes	Must satisfy Cognito password policy (8+ chars, upper, lower, digit).

Response: 200 OK

```
{
```

```
"status": "confirmed"
}
```

Errors:

Status	Condition
400	Missing email, code, or password. Code mismatch or expired. Invalid password format.
429	Rate limit exceeded.

GET /auth/me

Returns the DynamoDB profile for the currently authenticated user. Requires a valid Cognito ID token.

Handler: src/handlers/auth.js → exports.me

Auth: Cognito authorizer (any role, must be ACTIVE)

Request parameters: None

Headers:

```
Authorization: Bearer <idToken>
```

Response: 200 OK

```
{
  "user": { "<UserProfile>" }
}
```

Errors:

Status	Condition
401	Missing or invalid ID token.
403	Account is not ACTIVE.

User Management

All user management routes require an active manager session.

GET /users

Returns all user profiles sorted newest-updated first.

Handler: src/handlers/content.js → exports.listUsers

Auth: Cognito + role=manager, status=ACTIVE

Request parameters: None**Response: 200 OK**

```
{
  "users": [ { "<UserProfile>" } ]
}
```

Errors:

Status	Condition
401	Missing or invalid ID token.
403	User is not an active manager.

PUT /users/{userId}

Updates the role and status for a confirmed user. Cannot be used to set status to PENDING_CONFIRMATION. A manager cannot modify their own account to demote or suspend themselves.

Handler: src/handlers/content.js → exports.updateUser**Auth:** Cognito + role=manager, status=ACTIVE**Path parameters:**

Parameter	Type	Required	Description
userId	string	yes	Cognito sub of the target user.

Request body (JSON):

Field	Type	Required	Valid values
role	string	yes	"rep" or "manager"
status	string	yes	"ACTIVE" or "SUSPENDED"

Example request:

```
{
  "role": "manager",
  "status": "ACTIVE"
}
```

Response: 200 OK

```
{
  "user": { "<UserProfile>" }
}
```

Errors:

Status	Condition
400	Missing or invalid role / status. Target user is PENDING_CONFIRMATION. Manager attempted to demote or suspend their own account.
403	Caller is not an active manager.
404	userId does not exist.

Dashboard

GET /dashboard

Returns aggregated analytics for all exam sessions. Supports optional scenario filtering.

Handler: src/handlers/content.js → exports.getDashboard

Auth: Cognito + role=manager, status=ACTIVE

Query parameters:

Parameter	Type	Required	Description
scenarioId	string	no	Filter analytics to a specific scenario. Omit or pass "ALL" for all scenarios.

Response: 200 OK

```
{
  "generatedAt": "2026-05-27T10:00:00.000Z",
  "selectedScenarioId": "ALL",
  "passScore": 80,
  "summary": {
    "totalAttempts": 42,
    "activeAttempts": 2,
    "completedAttempts": 40,
    "evaluatedAttempts": 37,
    "avgSuccessScore": 84,
    "passRate": 62,
    "repsCount": 8,
    "repsEvaluated": 7,
    "needsEvaluation": 3
  },
  "scenarios": [
    {
      "scenarioId": "scenario_...",
      "title": "Q2 renewal pressure test",
      "attempts": 18,
      "avgScore": 86,
      "passRate": 67
    }
  ],
  "reps": [
```

```
{
  "userId": "a1b2c3d4-...",
  "name": "Riley Rep",
  "email": "riley@salesops.ai",
  "attempts": 5,
  "latestScore": 91,
  "averageScore": 88,
  "bestScore": 95,
  "passRate": 75,
  "completionRate": 100,
  "evaluatedAttempts": 4,
  "needsEvaluation": 1,
  "lastAttemptDate": "2026-05-24T09:00:00.000Z",
  "coachingFocus": "Add exact timeline commitments"
},
{
  "scoreBands": [
    { "label": "Passed", "min": 80, "max": 100, "count": 23, "color": "#2d6d5f", "percent": 62 },
    { "label": "Needs coaching", "min": 60, "max": 79, "count": 9, "color": "#d7a13e", "percent": 24 },
    { "label": "At risk", "min": 0, "max": 59, "count": 3, "color": "#b85b3e", "percent": 8 },
    { "label": "Not evaluated", "min": null, "max": null, "count": 2, "color": "#9a8f7d", "percent": 5 }
  ]
}
```

Implementation note: The dashboard Lambda performs a full table scan on ExamSessions, Users, and Scenarios tables on every call. This is acceptable at current scale but will need GSIs or pre-aggregated records for high-volume production use.

Errors:

Status	Condition
403	Caller is not an active manager.
500	EXAM_SESSIONS_TABLE_NAME environment variable not configured.

Personas

GET /personas

Returns all personas sorted newest-updated first.

Handler: src/handlers/content.js → exports.listPersonas

Auth: Cognito + role=manager, status=ACTIVE

Response: 200 OK

```
{
  "personas": [ { "<Persona>" } ]
}
```

POST /personas

Creates a new persona with status ACTIVE. The personaId is generated server-side.

Handler: src/handlers/content.js → exports.createPersona

Auth: Cognito + role=manager, status=ACTIVE

Request body (JSON):

Field	Type	Required	Constraints
name	string	yes	Trimmed. Cannot be empty.
description	string	no	Trimmed. Defaults to empty string.
behaviorNotes	string	no	Trimmed. Defaults to empty string.

Example request:

```
{
  "name": "Frustrated finance manager",
  "description": "Controls budget approval and renewal decisions",
  "behaviorNotes": "Direct and time-pressured. Expects ownership and a clear next step."
}
```

Response: 201 Created

```
{
  "persona": { "<Persona>" }
}
```

Errors:

Status	Condition
400	name is missing or empty.
403	Caller is not an active manager.

PUT /personas/{personaId}

Replaces all mutable fields on an existing persona. The personaId must already exist.

Handler: src/handlers/content.js → exports.updatePersona

Auth: Cognito + role=manager, status=ACTIVE

Path parameters:

Parameter	Type	Required
personaId	string	yes

Request body (JSON):

Field	Type	Required	Constraints
name	string	yes	Cannot be empty.
description	string	no	Defaults to empty string.
behaviorNotes	string	no	Defaults to empty string.
status	string	no	Preserved from previous value if not provided.

Response: 200 OK

```
{
  "persona": { "<Persona>" }
}
```

Errors:

Status	Condition
400	personaId missing in path or name is empty.
403	Caller is not an active manager.
404	Persona not found.

Scenarios

GET /scenarios

Returns all scenarios (DRAFT, PUBLISHED, and ARCHIVED) sorted newest-updated first. Includes all embedded issues.

Handler: src/handlers/content.js → exports.listScenarios

Auth: Cognito + role=manager, status=ACTIVE

Response: 200 OK

```
{
  "scenarios": [ { "<Scenario>" } ]
}
```

POST /scenarios

Creates a scenario draft. The scenarioId is generated server-side. Status is always DRAFT on creation.

Handler: src/handlers/content.js → exports.createScenario

Auth: Cognito + role=manager, status=ACTIVE

Request body (JSON):

Field	Type	Required	Constraints
title	string	yes	Cannot be empty.
description	string	no	Defaults to empty string.
personaIds	string[]	no	Array of existing persona IDs. Defaults to [].
issueCount	integer	no	1-20 inclusive. Defaults to 5.

Example request:

```
{
  "title": "Q2 renewal pressure test",
  "description": "Rep handles renewal friction and billing concerns",
  "personaIds": ["persona_abc123"],
  "issueCount": 5
}
```

Response: 201 Created

```
{
  "scenario": { "<Scenario>" }
}
```

Errors:

Status	Condition
400	title missing. issueCount outside 1-20 range.
403	Caller is not an active manager.

PUT /scenarios/{scenarioId}

Updates all mutable fields on an existing scenario. Does not change issues array — use POST /scenarios/{scenarioId}/issues/generate to replace issues.

Handler: src/handlers/content.js → exports.updateScenario

Auth: Cognito + role=manager, status=ACTIVE

Path parameters:

Parameter	Type	Required
scenarioId	string	yes

Request body (JSON):

Field	Type	Required	Constraints
title	string	yes	Cannot be empty.

Field	Type	Required	Constraints
description	string	no	
personaIds	string[]	no	Replaces existing list.
issueCount	integer	no	1-20. Preserved from previous if not provided.
status	string	no	Must be DRAFT, PUBLISHED, or ARCHIVED. Preserved from previous if not provided.

Response: 200 OK

```
{
  "scenario": { "<Scenario>" }
}
```

Errors:

Status	Condition
400	title missing. issueCount out of range. Invalid status value.
403	Caller is not an active manager.
404	Scenario not found.

POST /scenarios/{scenarioId}/publish

Sets scenario status to PUBLISHED. Requires at least one personaId to be set.

Handler: src/handlers/content.js → exports.publishScenario

Auth: Cognito + role=manager, status=ACTIVE

Path parameters:

Parameter	Type	Required
scenarioId	string	yes

Request body: None

Response: 200 OK

```
{
  "scenario": { "<Scenario>" }
}
```

Errors:

Status	Condition
400	Scenario has no linked personas.
403	Caller is not an active manager.
404	Scenario not found.

POST /scenarios/{scenarioId}/clone

Creates a full deep copy of the scenario as a new DRAFT. All embedded issues are given new `issueId` values. The title becomes `<original title> copy`.

Handler: `src/handlers/content.js → exports.cloneScenario`

Auth: Cognito + `role=manager, status=ACTIVE`

Path parameters:

Parameter	Type	Required
<code>scenarioId</code>	string	yes

Request body: None

Response: 201 Created

```
{
  "scenario": { "<Scenario>" }
}
```

Errors:

Status	Condition
403	Caller is not an active manager.
404	Source scenario not found.

POST /scenarios/{scenarioId}/archive

Sets scenario status to ARCHIVED. Archived scenarios no longer appear in the rep exam scenario list.

Handler: `src/handlers/content.js → exports.archiveScenario`

Auth: Cognito + `role=manager, status=ACTIVE`

Path parameters:

Parameter	Type	Required
<code>scenarioId</code>	string	yes

Request body: None

Response: 200 OK

```
{
  "scenario": { "<Scenario>" }
}
```

Errors:

Status	Condition
403	Caller is not an active manager.
404	Scenario not found.

POST /scenarios/{scenarioId}/issues/generate

Calls the OpenAI Responses API with a strict JSON schema to generate issueCount inbox issues for the scenario. Replaces any existing issues. Falls back to demo issues if OpenAI fails.

Handler: src/handlers/content.js → exports.generateScenarioIssues

Auth: Cognito + role=manager, status=ACTIVE

Lambda config: Timeout 25 s, Memory 256 MB

Path parameters:

Parameter	Type	Required
scenarioId	string	yes

Request body: None

External dependency: AWS Secrets Manager secret salesops/dev/llm-api-keys must contain OPENAI_API_KEY. OpenAI Responses API endpoint: https://api.openai.com/v1/responses. Model: controlled by OPENAI_MODEL environment variable (default gpt-5-mini).

Response: 200 OK

```
{
  "scenario": { "<Scenario with issues populated>" },
  "generationSource": "OPENAI",
  "warning": ""
}
```

Field	Type	Description	
generationSource	"OPENAI" \	"DEMO"	"DEMO" when OpenAI failed and demo fallback was used.
warning	string	Non-empty message when demo issues were generated.	

Demo fallback behavior: If the OpenAI call fails with a 5xx error, the Lambda generates `issueCount` placeholder issues using an internal pattern table. These issues are marked with `generationSource="DEMO"` and a non-empty `warning`. The Lambda still returns 200. Demo issues are clearly distinguishable from production issues by their subject and message patterns.

Errors:

Status	Condition
400	Scenario is not PUBLISHED. No personas linked.
403	Caller is not an active manager.
404	Scenario not found. One or more linked personas not found.
502	OpenAI returned a 4xx error (not retried via demo).

PUT /scenarios/{scenarioId}/issues/{issueId}

Replaces `customerName`, `subject`, `message`, and `difficulty` on a single generated issue. The issue must already exist in the scenario's `issues` array.

Handler: `src/handlers/content.js` → `exports.updateScenarioIssue`

Auth: Cognito + `role=manager`, `status=ACTIVE`

Path parameters:

Parameter	Type	Required
<code>scenarioId</code>	string	yes
<code>issueId</code>	string	yes

Request body (JSON):

Field	Type	Required	Constraints
<code>customerName</code>	string	yes	Cannot be empty.
<code>subject</code>	string	yes	Cannot be empty.
<code>message</code>	string	yes	Cannot be empty.
<code>difficulty</code>	string	yes	"EASY", "MEDIUM", or "HARD" (case-insensitive on input, stored uppercase).

Response: 200 OK

```
{
  "scenario": { "<Scenario>" },
  "issue": { "<ScenarioIssue>" }
}
```

Errors:

Status	Condition
400	Any required field is missing. Invalid difficulty value.
403	Caller is not an active manager.
404	Scenario not found. Issue not found inside scenario.

Exam — Representative Routes

All exam routes require `role=rep` and `status=ACTIVE`.

GET /exam/scenarios

Returns all scenarios that are `PUBLISHED` and have at least one generated issue. Returns a summary view without the full issue list. Also returns the exam duration in seconds.

Handler: `src/handlers/content.js → exports.listExamScenarios`

Auth: `Cognito + role=rep, status=ACTIVE`

Response: 200 OK

```
{
  "scenarios": [
    {
      "scenarioId": "scenario_...",
      "title": "Q2 renewal pressure test",
      "description": "Rep handles renewal friction and billing concerns",
      "issueCount": 5,
      "generatedIssueCount": 5
    }
  ],
  "durationSeconds": 180
}
```

Field	Type	Description
durationSeconds	integer	Always 180. Fixed constant from <code>examDurationSeconds</code> .
generatedIssueCount	integer	Actual count of issues in <code>scenario.issues</code> . Should equal <code>issueCount</code> when generation is complete.

Errors:

Status	Condition
403	Caller is not an active rep.

POST /exam/sessions

Creates a new exam session for the authenticated rep. Copies all scenario issues into ExamSessions table. Schedules delayed issue releases via SQS. The first issue (orderIndex=0) is immediately visible. Later issues are released at evenly spaced intervals over the 180-second window.

Handler: src/handlers/content.js → exports.createExamSession

Auth: Cognito + role=rep, status=ACTIVE

Request body (JSON):

Field	Type	Required	Constraints
scenarioId	string	yes	Must be a PUBLISHED scenario with at least one generated issue.

Example request:

```
{
  "scenarioId": "scenario_abc123"
}
```

Issue release timing formula:

```
delaySeconds(index) = floor(180 * index / totalIssues)
releaseAt = startedAt + delaySeconds
```

Issue at index=0 has delaySeconds=0 and is visible immediately. SQS DelaySeconds is capped at 900 seconds (SQS maximum).

DynamoDB writes:

- One META record (recordId = "META") per session.
- One ISSUE#<issueId> record per issue.

SQS writes:

- One SQS message per issue with delaySeconds > 0. Message body: { "sessionId": "...", "issueId": "...", "releaseAt": "..." }.

Response: 201 Created

```
{
  "session": { "<ExamSession>" }
}
```

Errors:

Status	Condition
400	scenarioId missing. Scenario is not PUBLISHED. Scenario has no generated issues.
403	Caller is not an active rep.
404	Scenario not found.

Status	Condition
500	EXAM_ISSUE_RELEASE_QUEUE_URL environment variable not configured.

GET /exam/sessions/{sessionId}/pulse

Returns the current state of the exam session: remaining time, visible issues, and their responses. Also reveals any issues whose releaseAt time has passed but whose SQS delivery has not yet fired.

Handler: src/handlers/content.js → exports.getExamSessionPulse

Auth: Cognito + role=rep, status=ACTIVE, session must belong to caller

Path parameters:

Parameter	Type	Required
sessionId	string	yes

Side effect: If any issue has releaseAt <= now and isVisible=false, the Lambda calls PutItem to mark it visible before returning. This makes issue release resilient to SQS delivery delays.

Response: 200 OK

```
{
  "session": {
    "sessionId": "session_...",
    "scenarioId": "scenario_...",
    "title": "Q2 renewal pressure test",
    "description": "Rep handles renewal friction",
    "durationSeconds": 180,
    "totalIssues": 5,
    "startedAt": "2026-05-27T10:00:00.000Z",
    "endsAt": "2026-05-27T10:03:00.000Z",
    "status": "ACTIVE",
    "remainingSeconds": 142
  },
  "issues": [ { "<ExamIssue>" } ]
}
```

Field	Type	Description	
session.status	"ACTIVE" \	"ENDED"	Derived from remainingSeconds > 0.
session.remainingSeconds	integer	Seconds until endsAt. Zero when the exam is over.	
issues	ExamIssue[]	Only visible issues, sorted by orderIndex.	

Errors:

Status	Condition
403	Session belongs to a different rep.
404	Session not found.

POST /exam/sessions/{sessionId}/issues/{issueId}/responses

Appends a response to a visible, non-done issue. The exam session must still be active (not expired or ENDED).

Handler: src/handlers/content.js → exports.submitExamIssueResponse

Auth: Cognito + role=rep, status=ACTIVE, session must belong to caller

Path parameters:

Parameter	Type	Required
sessionId	string	yes
issueId	string	yes

Request body (JSON):

Field	Type	Required	Constraints
message	string	yes	1-4000 characters after trimming. Cannot be empty.

Response: 201 Created

```
{
  "issue": { "<ExamIssue>" }
}
```

The returned issue includes all existing responses plus the newly appended one.

Errors:

Status	Condition
400	message missing or empty. Message exceeds 4000 characters. Issue is not visible yet. Issue is already DONE. Exam session has ended.
403	Session belongs to a different rep.
404	Session not found. Issue not found.

POST /exam/sessions/{sessionId}/issues/{issueId}/done

Marks an issue as DONE. At least one response must have been submitted. The exam session must still be active.

Handler: src/handlers/content.js → exports.markExamIssueDone

Auth: Cognito + role=rep, status=ACTIVE, session must belong to caller

Path parameters:

Parameter	Type	Required
sessionId	string	yes
issueId	string	yes

Request body: None

Response: 200 OK

```
{  
  "issue": { "<ExamIssue>" }  
}
```

The returned issue has status="DONE" and a populated doneAt timestamp.

Errors:

Status	Condition
400	No responses submitted yet. Issue is already DONE. Issue is not visible. Exam session has ended.
403	Session belongs to a different rep.
404	Session not found. Issue not found.

POST /exam/sessions/{sessionId}/evaluation

Creates the AI evaluation for a completed session. Idempotent: if an evaluation already exists it is returned immediately without calling OpenAI again.

Handler: src/handlers/content.js → exports.createExamEvaluation

Auth: Cognito + role=rep, status=ACTIVE, session must belong to caller

Lambda config: Timeout 30 s, Memory 256 MB

Path parameters:

Parameter	Type	Required
sessionId	string	yes

Request body: None

Behavior:

1. If an EVALUATION record already exists, return it immediately with 200.
2. If the session is still active, return 400.

3. Call OpenAI Responses API with exam context, rubric definition, and all visible issue/response pairs.
4. Normalize scores (AI may return 0-5 or 0-100 scales; all are normalized to 0-100).
5. Write EVALUATION record and update META record `sessionStatus` to "ENDED".

OpenAI JSON schema enforced: All five rubric keys, overall notes/strengths/growth/practice arrays, and one per-issue entry per visible issue. Missing rep responses receive low scores.

Response: 200 OK (existing evaluation) or **201 Created** (new evaluation)

```
{
  "evaluation": { "<ExamEvaluation>" }
}
```

Errors:

Status	Condition
400	Exam session is still active. Session has no issues.
403	Session belongs to a different rep.
404	Session not found.
502	OpenAI evaluation request failed.

GET /exam/sessions/{sessionId}/evaluation

Returns the persisted evaluation for a completed session. Does not call OpenAI.

Handler: `src/handlers/content.js` → `exports.getExamEvaluation`

Auth: Cognito + `role=rep`, `status=ACTIVE`, session must belong to caller

Path parameters:

Parameter	Type	Required
<code>sessionId</code>	string	yes

Response: 200 OK

```
{
  "evaluation": { "<ExamEvaluation>" }
}
```

Errors:

Status	Condition
403	Session belongs to a different rep.
404	Session not found. Evaluation not yet created.

Internal — SQS Consumer

releaseExamIssue (SQS trigger)

Marks scheduled exam issues visible when their SQS delay expires. This function is not callable over HTTP — it is triggered by the ExamIssueReleaseQueue SQS queue.

Handler: src/handlers/content.js → exports.releaseExamIssue

Trigger: SQS event, batch size 10, ReportBatchItemFailures enabled

SQS message body:

```
{
  "sessionId": "session_...",
  "issueId": "issue_..."
}
```

Field	Type	Required
sessionId	string	yes
issueId	string	yes

Behavior:

1. Parse sessionId and issueId from the SQS message body.
2. Read the ISSUE#<issueId> record from ExamSessions.
3. If the record exists and isVisible=false, overwrite it with isVisible=true, issueStatus="VISIBLE", and visibleAt=now.
4. If any message in the batch fails, add its messageId to batchItemFailures so SQS retries only failed messages.

Return:

```
{
  "batchItemFailures": []
}
```

On partial batch failure:

```
{
  "batchItemFailures": [
    { "itemIdentifier": "<messageId>" }
  ]
}
```

DynamoDB Table Schemas

Users

Attribute	DynamoDB type	Key
userId	String	Partition key
email	String	
emailLower	String	GSI EmailIndex partition key
fullName	String	
role	String	"rep" or "manager"
status	String	"PENDING_CONFIRMATION", "ACTIVE", or "SUSPENDED"
createdAt	String	ISO 8601
updatedAt	String	ISO 8601

GSI: EmailIndex on emailLower (projection: ALL). Used during signup and signin to find profiles by email.

Personas

Attribute	DynamoDB type	Key
personaId	String	Partition key
name	String	
description	String	
behaviorNotes	String	
status	String	Always "ACTIVE"
createdAt	String	
updatedAt	String	

Scenarios

Attribute	DynamoDB type	Key
scenarioId	String	Partition key
title	String	
description	String	
personaIds	List of String	
issueCount	Number	
issues	List of Map	Embedded ScenarioIssue objects
issuesGeneratedAt	String	
generationSource	String	"OPENAI", "DEMO", or empty
generationWarning	String	
status	String	"DRAFT", "PUBLISHED", or "ARCHIVED"
createdAt	String	

Attribute	DynamoDB type	Key
updatedAt	String	

ExamSessions

Composite key table. One session produces multiple DynamoDB items.

Attribute	DynamoDB type	Key
sessionId	String	Partition key
recordId	String	Sort key

Record types:

recordId value	Content
"META"	Session metadata, userId, scenarioId, timing, status
"ISSUE#<issueId>"	Per-issue data, responses list, visibility, done state
"EVALUATION"	AI evaluation result

Querying a full session uses KeyConditionExpression: sessionId = :sessionId which retrieves all record types for the session in one request.

Environment Variables

Variable	Handler file	Description
USER_POOL_CLIENT_ID	auth.js	Cognito app client ID
USERS_TABLE_NAME	auth.js, content.js	DynamoDB Users table name
PERSONAS_TABLE_NAME	content.js	DynamoDB Personas table name
SCENARIOS_TABLE_NAME	content.js	DynamoDB Scenarios table name
EXAM_SESSIONS_TABLE_NAME	content.js	DynamoDB ExamSessions table name
EXAM_ISSUE_RELEASE_QUEUE_URL	content.js	SQS queue URL for issue release
LLM_SECRET_NAME	content.js	Secrets Manager secret name. Default: salesops/dev/llm-api-keys
OPENAI_MODEL	content.js	OpenAI model ID. Default: gpt-5-mini
SERVICE_NAME	health.js	Returned in health check. Default: salesops-ai
STAGE_NAME	health.js	Returned in health check. Default: dev
AWS_REGION	All	AWS region. Set automatically by the Lambda runtime.

Error Reference

All error responses use the format:

```
{ "message": "<string>" }
```

Auth handler error map:

Cognito exception	HTTP status	Message
UsernameExistsException	409	Account already exists for this email.
InvalidPasswordException / InvalidParameterException	400	Error message from Cognito.
CodeMismatchException / ExpiredCodeException	400	Confirmation code is invalid or expired.
UserNotConfirmedException	403	Confirm your email before signing in.
LimitExceededException / TooManyRequestsException	429	Too many attempts. Try again later.
NotAuthorizedException / UserNotFoundException	401	Email or password is incorrect.
SyntaxError (bad JSON body)	400	Request body must be valid JSON.

Content handler error map:

Condition	HTTP status	Message
Bad JSON body	400	Request body must be valid JSON.
Missing auth	401	Missing authenticated user.
Not manager	403	Manager access required.
Not rep	403	Rep access required.
Not active	403	Active user required.
Session ownership	403	Exam session access denied.
Not found	404	Resource-specific not found message.
OpenAI 4xx	502	OpenAI-specific error message.
OpenAI refusal	502	OpenAI refused to complete this request.
OpenAI invalid JSON	502	OpenAI returned malformed data.
Uncaught 5xx	500	Content request failed.